HW10 Solutions

```
# Load dataset
df <- read.csv("HW10_data.csv")
```

1)1)

```
# Filter points where |x| < 0.5
subset <- df[abs(df$x) < 0.5, ]

# Compute estimator
m_hat_0 <- mean(subset$y)

cat("m_hat(0):", m_hat_0, "\n")
cat("Number of points used:", nrow(subset), "\n")
```

```
m_hat(0): 0.007849275
Number of points used: 15
```

1)2)

```
bias <- mean(subset$x)
cat("Bias:", bias, "\n")
```

```
Bias: -0.01079084
```

1)4)

```
x <- df$x
y <- df$y
n <- length(x)
h <- 0.5

m_hat <- numeric(n)

# Compute m_hat(x_i)
for (i in 1:n) {
  mask <- abs(x - x[i]) < h
  m_hat[i] <- mean(y[mask])
}

# Compute sigma^2 estimate
sigma_hat_sq <- sum((y - m_hat)^2) / (n - 1)

cat("Estimated sigma^2:", sigma_hat_sq, "\n")
```

```
Estimated sigma^2: 0.04202859
```

1)5)

```
# N0
N0 <- nrow(df[abs(df$x) < 0.5, ])

# m_hat(0)
m_hat_0 <- mean(df[abs(df$x) < 0.5, ]$y)

# CI
lower <- m_hat_0 - 1.96 * sqrt(sigma_hat_sq * (1 + 1/N0))
upper <- m_hat_0 + 1.96 * sqrt(sigma_hat_sq * (1 + 1/N0))

cat("95% CI:", lower, upper, "\n")
```

```
95% CI: -0.4071457 0.4228443
```

2)1)

```
# Sort by x
df <- df[order(df$x), ]
```

```
x <- df$x
y <- df$y
n <- length(x)

best_r <- NA
min_sse <- Inf

for (i in 2:n) {
  r <- (x[i-1] + x[i]) / 2

  left <- y[x <= r]
  right <- y[x > r]

  if (length(left) == 0 || length(right) == 0) next

  m1 <- mean(left)
  m2 <- mean(right)

  sse <- sum((left - m1)^2) + sum((right - m2)^2)

  if (sse < min_sse) {
    min_sse <- sse
    best_r <- r
  }
}

cat("Optimal r:", best_r, "\n")
cat("Minimum SSE:", min_sse, "\n")
```

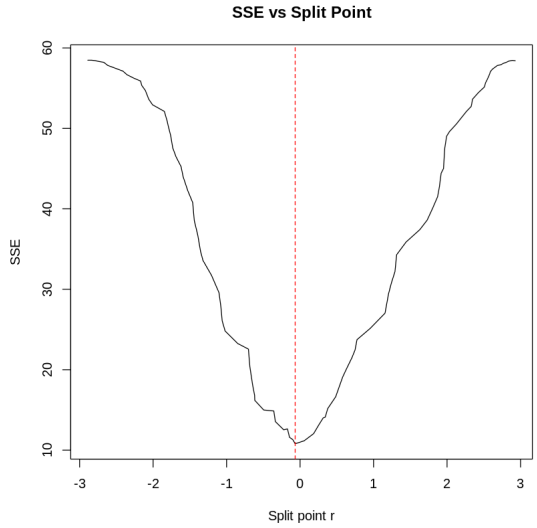
```
Optimal r: -0.06695331
Minimum SSE: 10.79163
```

2)2)

```
x <- data$x
y <- data$y
n <- length(x)

# Step 1: Sort data
order_idx <- order(x)
x <- x[order_idx]
y <- y[order_idx]

# Step 2: Candidate split points (MIDPOINTS - recommended)
candidates <- (x[-1] + x[-n]) / 2

# Initialize
sse_values <- numeric(length(candidates))

# Step 3: Compute SSE for each split
for (j in 1:length(candidates)) {
  r <- candidates[j]

  left  <- y[x <= r]
  right <- y[x > r]

  m1 <- mean(left)
  m2 <- mean(right)

  sse_values[j] <- sum((left - m1)^2) + sum((right - m2)^2)
}

# Step 4: Find optimal split
best_idx <- which.min(sse_values)
r_opt <- candidates[best_idx]
min_sse <- sse_values[best_idx]

# Step 5: Compute final m1 and m2
m1_opt <- mean(y[x <= r_opt])
m2_opt <- mean(y[x > r_opt])

# Step 6: Print results clearly
cat("Optimal split (r):", r_opt, "\n")
cat("Minimum SSE:", min_sse, "\n\n")

cat("Final regression function:\n")
cat("m(x) = ", m1_opt, " if x <=", r_opt, "\n")
cat("m(x) = ", m2_opt, " if x >", r_opt, "\n")

# Optional: Plot SSE vs r
```

```
# Optional: Plot SSE vs r
plot(candidates, sse_values, type = "l",
     xlab = "Split point r",
     ylab = "SSE",
     main = "SSE vs Split Point")
abline(v = r_opt, col = "red", lty = 2)
```

```
Optimal split (r): -0.06695331
Minimum SSE: 10.79163

Final regression function:
m(x) =  -0.717591  if x <= -0.06695331
m(x) =  0.6636164  if x > -0.06695331
```



2)3)

```
x_val <- 0.5

if (x_val <= r_opt) {
  prediction <- m1_opt
} else {
  prediction <- m2_opt
}

cat("Predicted value at x = 0.5:", prediction, "\n")
```

```
Predicted value at x = 0.5: 0.6636164
```

2)4)

```
set.seed(123)

B <- 500   # number of bootstrap samples
n <- length(x)

boot_preds <- numeric(B)

for (b in 1:B) {

  # Step 1: Bootstrap sample
  idx <- sample(1:n, n, replace = TRUE)
  x_boot <- x[idx]
  y_boot <- y[idx]

  # Step 2: Sort
  order_idx <- order(x_boot)
  x_boot <- x_boot[order_idx]
  y_boot <- y_boot[order_idx]

  # Step 3: Candidate splits (midpoints)
  candidates <- (x_boot[-1] + x_boot[-n]) / 2

  sse_values <- numeric(length(candidates))

  # Step 4: Find best split
  for (j in 1:length(candidates)) {
    r <- candidates[j]
```

```
    left  <- y_boot[x_boot <= r]
    right <- y_boot[x_boot > r]

    if (length(left) == 0 || length(right) == 0) next

    m1 <- mean(left)
    m2 <- mean(right)

    sse_values[j] <- sum((left - m1)^2) + sum((right - m2)^2)
  }

  best_idx <- which.min(sse_values)
  r_boot <- candidates[best_idx]

  # Step 5: Compute prediction at x = 0.5
  m1_boot <- mean(y_boot[x_boot <= r_boot])
  m2_boot <- mean(y_boot[x_boot > r_boot])

  if (0.5 <= r_boot) {
    boot_preds[b] <- m1_boot
  } else {
    boot_preds[b] <- m2_boot
  }
}

# Step 6: Variance estimate
var_boot <- var(boot_preds)

cat("Bootstrap variance of m(0.5):", var_boot, "\n")
```

```
Bootstrap variance of m(0.5): 0.1215863
```

2)5)

```
# prediction from earlier
x_val <- 0.5

if (x_val <= r_opt) {
  m_hat <- m1_opt
} else {
  m_hat <- m2_opt
}

# bootstrap variance already computed as var_boot

# CI
lower <- m_hat - 1.96 * sqrt(var_boot)
upper <- m_hat + 1.96 * sqrt(var_boot)

cat("95% CI for m(0.5):", lower, upper, "\n")
```

```
95% CI for m(0.5): -0.01982047 1.347053
```